

SELECCIÓN DE FRAMEWORK DEL PROYECTO

PROYECTO: RENTA MOVIL LOCATION

INTEGRANTES:

LAURA VANESSA PEREZ PERDOMO
DANNA VALENTINA BARRIOS PENAGOS

INSTRUCTOR:

CARLOS JULIO CADENA

TECNOLOGO EN ANALISIS Y DESARROLLO DE SOFTWARE

FICHA: 3145555

SERVICIO NACIONAL DE APRENDIZAJE

CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS

TABLA DE CONTENIDO

1. Introducción	4
1.1 Propósito del documento.....	4
1.2 Alcance	4
1.3 Relación con el proyecto formativo.....	4
2. Análisis de Insumos del Proyecto	5
2.1 Análisis del SRS	5
2.2 Análisis del modelado.....	6
2.3 Análisis de mockups	6
2.4 Análisis de interacciones.....	7
3. Selección del Framework Front-End	9
3.1 Alternativas evaluadas	9
3.2 Framework seleccionado	9
3.3 Justificación técnica.....	10
3.4 Ventajas y limitaciones	10
4. Definición de la Arquitectura Front-End	12
4.1 Tipo de arquitectura seleccionada.....	12
4.2 Estructura del proyecto (carpetas y módulos).....	12
4.3 Flujo de la aplicación	13
4.4 Manejo de estados.....	13
5. Definición de Patrones de Diseño.....	15
5.1 Patrones seleccionados.....	15
5.2 Justificación de uso	15
5.3 Ejemplos de aplicación en el proyecto.....	16
5.4 Anti-patrones a evitar.....	17
6. Integración de la Solución Front-End	18
6.1 Relación con el SRS.....	18
6.2 Relación con mockups e interacciones	18
6.3 Relación con el modelado	19
7. Conclusiones	20
8. Recomendaciones	20
9. Anexos (Opcional)	21
9.1 Diagrama de arquitectura.....	21
9.2 Stack Tecnológico completo.....	21

9.3 Clasificación de patrones de diseño usados en el proyecto	22
---	----

1. Introducción

1.1 Propósito del documento

Este documento tiene como propósito definir y justificar las decisiones técnicas relacionadas con la selección del framework front-end, la arquitectura del proyecto y los patrones de diseño que se van a implementar en el desarrollo de Renta Movil Location. Está pensado para que el equipo de desarrollo tenga una guía clara de cómo estructurar el proyecto desde el inicio.

1.2 Alcance

El documento abarca el análisis de los artefactos ya construidos (SRS, mockups del Figma, modelado del sistema e interacciones), y con base en eso define qué framework se va a usar, cómo se va a organizar el proyecto y qué patrones de diseño se van a aplicar. No cubre temas de backend ni bases de datos.

1.3 Relación con el proyecto formativo

Este documento es parte de la formación del Tecnólogo en Análisis y Desarrollo de Software del SENA, ficha 3145555. Se desarrolla como actividad dentro del proyecto formativo Renta Movil Location, donde el equipo debe tomar decisiones técnicas reales sobre la construcción de una aplicación web y móvil para la gestión de alquiler de vehículos.

2. Análisis de Insumos del Proyecto

Antes de escoger el framework y la arquitectura, se revisaron todos los artefactos que ya se habían construido para el proyecto. Esto es importante porque la tecnología que se elija debe responder a las necesidades reales del sistema, no escogerse al azar.

2.1 Análisis del SRS

El documento SRS (Especificación de Requisitos de Software) de Renta Movil Location define un sistema que debe funcionar tanto como aplicación web como aplicación móvil (PWA — Progressive Web App) para dispositivos Android 13 y 14. A continuación se resumen los puntos más importantes que afectan la elección del front-end:

Funcionalidades principales que tiene el sistema:

- Registro e inicio de sesión con autenticación JWT y roles diferenciados (cliente, administrador, operador, supervisor).
- Catálogo de vehículos disponibles con filtros y calendario de disponibilidad en tiempo real.
- Módulo de reservas donde el usuario puede hacer, modificar y cancelar reservas.
- Módulo de pagos integrado con Wompi (PSE, tarjetas crédito/débito y Nequi).
- Generación y consulta de contratos en PDF.
- Panel administrativo para gestión de flota, contratos, pagos, mantenimientos y reportes.
- Generación de reportes en Word, PDF y Excel.
- Sistema de notificaciones para clientes y administradores.
- Aplicación móvil PWA compatible con Chrome y Firefox.

Requerimientos no funcionales importantes:

- Rendimiento: las pantallas críticas deben cargar en máximo 3 segundos.
- Seguridad: cumplimiento OWASP Top 10, cifrado SSL/TLS, tokenización de datos bancarios.
- Accesibilidad: cumplimiento de WCAG 2.1 nivel AA.
- Escalabilidad: arquitectura en N capas que separe presentación, lógica de negocio y acceso a datos.
- Compatibilidad: Chrome 146 y Firefox 134, Android 13 y 14.

2.2 Análisis del modelado

El modelado del sistema define las entidades principales con las que trabaja la aplicación. Estas entidades se relacionan directamente con los módulos y componentes visuales que debe manejar el front-end:

- Usuario (cliente, administrador, operador, supervisor)
- Vehículo (con estado de disponibilidad, tipo, características)
- Reserva (fechas, estado, relación con usuario y vehículo)
- Contrato (vinculado a una reserva)
- Pago (asociado a reserva, método de pago, estado)
- Mantenimiento (programado para vehículos)
- Reporte (generado por administradores)

Esto indica que el front-end debe manejar múltiples módulos con lógica propia y datos interrelacionados, lo que justifica usar una arquitectura modular con manejo de estado centralizado.

2.3 Análisis de mockups

Los mockups del proyecto (diseñados en Figma) comprenden un total de 30 pantallas, lo que indica una aplicación con complejidad visual considerable. Se identificaron los siguientes tipos de pantallas:

- Pantallas de autenticación: inicio de sesión y registro.
- Pantalla principal (home) con catálogo de vehículos.
- Pantallas de detalle de vehículo.
- Pantallas de reserva con calendario de disponibilidad.
- Pantallas de pago e integración con Wompi.
- Pantallas de historial de reservas y contratos.
- Panel administrativo con dashboards y tablas de gestión.
- Pantallas de reportes y notificaciones.
- Pantallas de perfil de usuario.

De este análisis se identificaron los siguientes componentes reutilizables:

- Tarjetas de vehículo (se repiten en catálogo, detalle y reservas).
- Formularios (registro, login, reserva, pago).
- Tablas de datos (historial, gestión administrativa).
- Calendarios de disponibilidad.
- Botones de acción y navegación.
- Modales y alertas de confirmación.
- Barra de navegación (navbar/sidebar).

La cantidad de pantallas y componentes reutilizables confirma que se necesita un framework que facilite la creación de componentes independientes y su reutilización a lo largo de toda la aplicación.

2.4 Análisis de interacciones

A partir de los mockups y el SRS se identificaron las siguientes interacciones principales que debe manejar el front-end:

Navegación entre vistas:

- Flujo de autenticación → home → detalle de vehículo → reserva → pago → confirmación.
- Panel administrativo con navegación entre módulos (flota, reservas, contratos, reportes).
- Navegación con historial del navegador (botón de regresar funcional).

Eventos del usuario:

- Selección de fechas en calendario de disponibilidad.
- Filtrado del catálogo de vehículos.
- Llenado y validación de formularios en tiempo real.
- Confirmación de pagos con integración a Wompi.
- Generación y descarga de contratos en PDF.
- Carga y visualización de reportes.

Flujo general de la aplicación:

El usuario (cliente) entra a la app → se registra o inicia sesión → ve el catálogo → selecciona un vehículo → hace una reserva → paga → recibe confirmación y puede descargar su contrato. El administrador tiene su propio flujo desde el panel de control para gestionar todo el sistema.

3. Selección del Framework Front-End

3.1 Alternativas evaluadas

Se evaluaron tres opciones de frameworks front-end teniendo en cuenta el tipo de aplicación (web + PWA móvil), la complejidad de la interfaz y los requerimientos del SRS:

Angular:

Framework completo de Google para aplicaciones web empresariales. Tiene todo integrado (enrutamiento, manejo de formularios, llamadas HTTP, etc.) pero su curva de aprendizaje es alta y puede ser muy complejo para equipos pequeños o en formación.

Vue.js:

Framework progresivo y amigable para principiantes. Es más sencillo que Angular y más estructurado que React básico, pero su ecosistema es menos amplio y hay menos recursos de aprendizaje disponibles en español comparado con React.

React:

Librería de JavaScript de Meta (Facebook) para construir interfaces de usuario basadas en componentes. Aunque técnicamente es una librería y no un framework completo, con herramientas como React Router, Axios y Zustand se convierte en una solución completa. Es el más usado a nivel mundial, tiene la comunidad más grande y miles de recursos de aprendizaje.

3.2 Framework seleccionado

React (con Vite como herramienta de construcción)

Para buscarlo en el navegador:

- Sitio oficial de React: react.dev
- Herramienta de construcción Vite: vitejs.dev
- Para enrutamiento: React Router (reactrouter.com)
- Para manejo de estado: Zustand (zustand-js.github.io) o Context API (viene incluido en React)
- Para peticiones HTTP: Axios (axios-http.com)
- Para estilos: Tailwind CSS (tailwindcss.com)

3.3 Justificación técnica

React fue seleccionado por las siguientes razones directamente relacionadas con el proyecto Renta Movil Location:

- La aplicación es una SPA (Single Page Application) y PWA (Progressive Web App): React está diseñado exactamente para este tipo de aplicaciones. Se puede convertir en PWA agregando un archivo de configuración (manifest.json y service worker) sin necesidad de cambiar de tecnología.
- Componentes reutilizables: Los 30 mockups del Figma mostraron muchos componentes que se repiten (tarjetas, formularios, tablas, botones). React permite crear estos componentes una sola vez y usarlos en múltiples pantallas.
- Compatibilidad con Android 13 y 14 vía PWA: Al ser una PWA, React corre directamente en el navegador del celular, sin necesitar instalación desde una tienda de aplicaciones. Funciona en Chrome 146 y Firefox 134 como lo exige el SRS.
- Manejo de estado para datos complejos: El sistema maneja múltiples entidades relacionadas (usuarios, vehículos, reservas, pagos). Con Zustand o Context API se puede manejar el estado global de la aplicación de forma ordenada.
- Integración con APIs REST: El SRS define que el backend usará una API REST. React con Axios permite consumir estas APIs fácilmente y manejar respuestas asíncronas.
- Comunidad y recursos: React es el framework front-end más usado en el mundo. Tiene miles de tutoriales, librerías adicionales y soporte de la comunidad, lo que facilita encontrar soluciones a problemas durante el desarrollo.

3.4 Ventajas y limitaciones

Ventajas de React para este proyecto:

- Fácil creación de componentes reutilizables.
- Gran ecosistema de librerías complementarias.
- Soporte nativo para PWA.
- Renderizado eficiente con Virtual DOM.
- Gran cantidad de recursos de aprendizaje en español.
- Compatible con todas las librerías de integración de pagos (Wompi tiene SDK para JavaScript).

Limitaciones a tener en cuenta:

- React por sí solo no incluye todo (se necesitan librerías adicionales para enrutamiento, manejo de estado, etc.).
- El manejo del estado global puede volverse complejo si no se organiza bien desde el inicio.
- Requiere configurar correctamente el service worker para que funcione como PWA.

4. Definición de la Arquitectura Front-End

4.1 Tipo de arquitectura seleccionada

Se seleccionó una arquitectura basada en componentes por módulos. Esto significa que la aplicación se divide en módulos según las funcionalidades del sistema (autenticación, catálogo, reservas, pagos, panel admin, etc.), y dentro de cada módulo se crean los componentes de React necesarios.

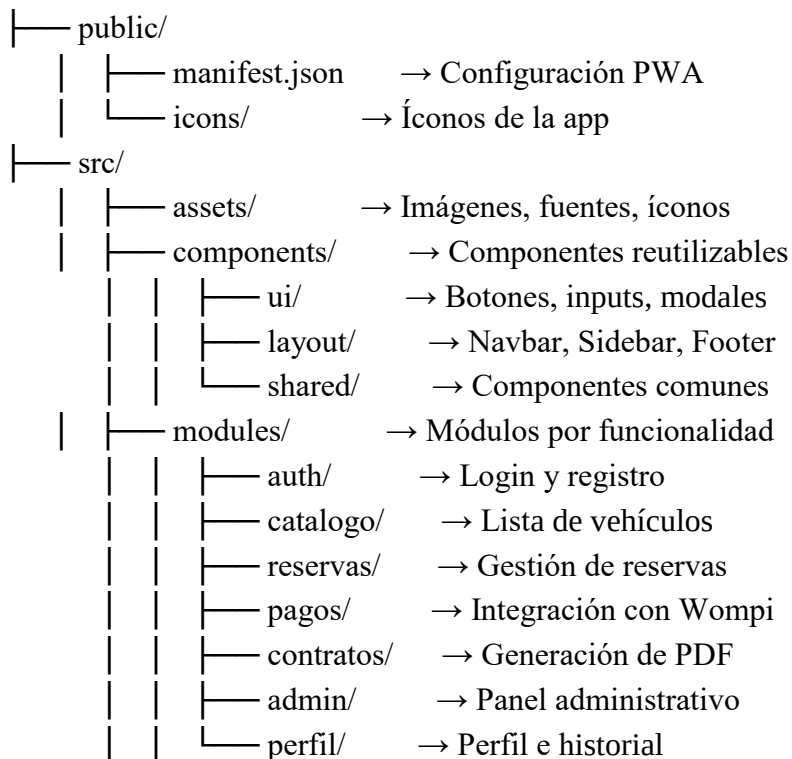
Esta arquitectura es ideal porque:

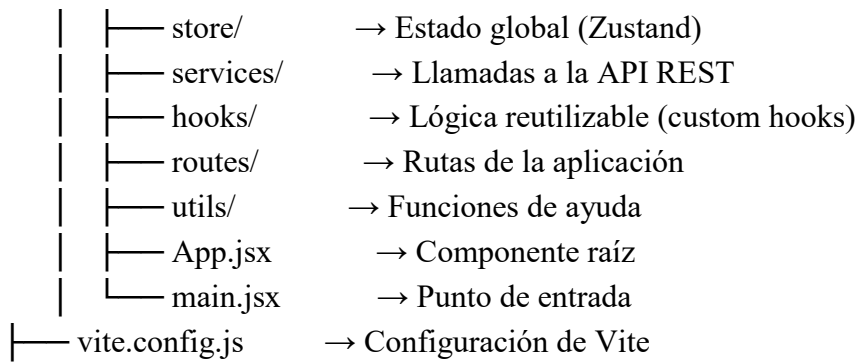
- Permite que cada módulo sea desarrollado y probado de forma independiente.
- Facilita el trabajo en equipo, ya que cada integrante puede trabajar en un módulo sin pisar el trabajo del otro.
- Es escalable: si en el futuro se necesita agregar un nuevo módulo, se puede hacer sin modificar lo que ya existe.

4.2 Estructura del proyecto (carpetas y módulos)

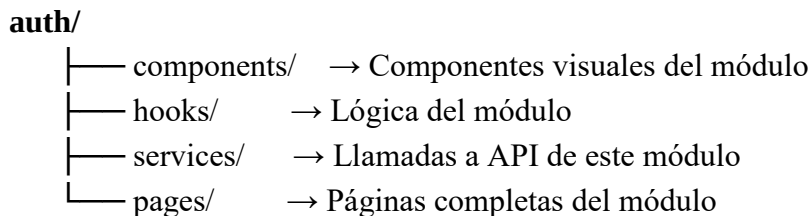
La estructura de carpetas propuesta para el proyecto es la siguiente:

renta-movil-location/





Cada módulo dentro de la carpeta modules/ sigue la misma estructura interna:



4.3 Flujo de la aplicación

El flujo general de la aplicación funciona así:

- El usuario abre la app (web o móvil vía PWA).
- Si no tiene sesión iniciada, React Router lo redirige automáticamente a la pantalla de login o registro.
- Después de autenticarse, el token JWT se guarda en el estado global (Zustand).
- Según el rol del usuario (cliente, administrador, operador), React Router muestra rutas diferentes.
- El cliente puede navegar por el catálogo, hacer reservas, pagar y ver su historial.
- El administrador accede al panel de control con todos los módulos de gestión.
- Cada módulo se comunica con el backend a través de llamadas HTTP usando Axios (capa de servicios).
- Los datos que se necesitan en múltiples partes de la app (usuario autenticado, carrito de reserva) se almacenan en el estado global.

4.4 Manejo de estados

El manejo de estado se divide en tres niveles:

Estado local (useState de React):

Se usa para datos que solo importan dentro de un componente, por ejemplo el valor de un campo de formulario o si un menú desplegable está abierto o cerrado.

Estado del módulo (Context o props):

Se usa cuando varios componentes dentro del mismo módulo necesitan compartir datos, por ejemplo los datos del vehículo seleccionado dentro del módulo de reservas.

Estado global (Zustand):

Se usa para datos que necesitan estar disponibles en toda la aplicación, como la información del usuario autenticado, el token JWT y las notificaciones. Zustand es simple, liviano y fácil de entender comparado con Redux.

5. Definición de Patrones de Diseño

5.1 Patrones seleccionados

Se seleccionaron los siguientes patrones de diseño para el proyecto. Se muestra el nombre formal del patrón y entre paréntesis cómo se conoce en el contexto de React:

- Composite → Componentes inteligentes y tontos (Smart/Dumb Components)
- Template Method → Custom Hooks
- Facade → Capa de Servicios (Service Layer)
- Strategy → Rutas Protegidas por rol (Protected Routes)
- Observer → Manejo de estado global (Zustand)

5.2 Justificación de uso

Cada patrón fue seleccionado por una razón concreta dentro del proyecto:

Composite → Componentes Inteligentes y Tontos:

El sistema tiene muchos componentes visuales (tarjetas, tablas, formularios) que pueden aparecer en varios módulos. Al separar la lógica de la presentación, el mismo componente visual puede usarse con datos diferentes sin repetir código. Un componente 'tonto' solo recibe datos y los muestra; un componente 'inteligente' obtiene los datos y se los pasa al tonto.

Template Method → Custom Hooks:

Hay lógica que se repite en varios módulos, como verificar si el usuario está autenticado, hacer llamadas a la API o manejar el estado de carga. Los custom hooks permiten extraer esta lógica a un lugar separado y reutilizarla sin copiar código. Ejemplo: `useAuth()` para manejar la autenticación, `useVehiculos()` para obtener la lista de vehículos.

Facade → Capa de Servicios:

Todas las llamadas al backend (API REST) se hacen desde la carpeta `services/`, no directamente desde los componentes. Esto centraliza la lógica de comunicación con el servidor y hace más fácil cambiar la URL de la API o agregar autenticación a las peticiones sin tocar cada componente.

Strategy → Rutas Protegidas:

El SRS exige autenticación JWT con roles diferenciados. Las rutas protegidas verifican si el usuario tiene sesión y el rol correcto antes de mostrar una página. Si no tiene permiso, lo redirigen al login automáticamente. Ejemplo: el panel administrativo solo lo pueden ver usuarios con rol 'administrador'.

Observer → Manejo de estado global (Zustand):

Zustand implementa el patrón Observer. Cuando el estado global cambia (por ejemplo, se confirma un pago o el usuario inicia sesión), todos los componentes que están 'observando' ese estado se actualizan automáticamente. Esto evita tener que pasar datos de componente en componente manualmente.

5.3 Ejemplos de aplicación en el proyecto

A continuación se muestran ejemplos concretos de cómo se aplica cada patrón en el proyecto Renta Movil Location:

Composite → Tarjeta de Vehículo (componente tonto):

Se crea un componente VehiculoCard que recibe por props el nombre, imagen, precio y disponibilidad del vehículo y los muestra. Este mismo componente se usa en el catálogo, en el panel de administración y en el historial de reservas, sin repetir el código visual.

Template Method → useReservas() (custom hook):

Un hook personalizado que encapsula la lógica para obtener las reservas del usuario, manejar el estado de carga y los errores. Cualquier componente que necesite mostrar reservas simplemente llama a useReservas() y obtiene los datos listos para usar.

Strategy → Ruta Protegida por rol:

La ruta /admin solo está disponible para usuarios con rol 'administrador'. Si un cliente intenta acceder directamente a esa URL, el sistema lo redirige automáticamente a su panel de cliente. Esto se implementa con un componente ProtectedRoute que envuelve todas las rutas del panel admin.

Facade → vehiculosService.js (capa de servicios):

Todas las peticiones relacionadas con vehículos (obtener lista, ver detalle, verificar disponibilidad) se hacen desde un archivo `vehiculosService.js`. Si el backend cambia la URL de la API, solo hay que actualizar ese archivo, no todos los componentes.

Observer → Estado global con Zustand:

Cuando el usuario confirma una reserva, el store de Zustand actualiza el estado global. Automáticamente todos los componentes que dependen de ese estado (contador de reservas en el navbar, historial, notificaciones) se actualizan sin necesidad de recargar la página.

5.4 Anti-patrones a evitar

También es importante saber qué NO hay que hacer para no crear problemas más adelante:

- Prop drilling excesivo: pasar datos de componente en componente en cadena larga. Si un dato se necesita en varios niveles, se debe usar el estado global (Zustand) o Context API.
- Lógica de negocio dentro de los componentes visuales: los componentes de presentación no deben hacer llamadas a la API. Eso va en los servicios o en los custom hooks.
- Componentes demasiado grandes: si un componente tiene más de 200 líneas de código, probablemente se deba dividir en componentes más pequeños.
- No manejar los estados de carga y error: cada llamada a la API puede tardar o fallar. Los componentes deben mostrar un indicador de carga mientras esperan y un mensaje de error si algo falla.
- Usar fetch directamente en los componentes en lugar de Axios con la capa de servicios.

6. Integración de la Solución Front-End

6.1 Relación con el SRS

Cada módulo del front-end responde directamente a los requerimientos funcionales del SRS:

- RF de autenticación → Módulo auth/ con rutas protegidas por JWT.
- RF de catálogo y disponibilidad → Módulo catalogo/ con componentes de tarjeta de vehículo y calendario.
- RF de reservas → Módulo reservas/ con formulario de reserva y gestión de estado.
- RF de pagos (Wompi) → Módulo pagos/ con integración al SDK de Wompi para PSE, tarjetas y Nequi.
- RF de contratos → Módulo contratos/ con generación y descarga de PDF.
- RF de panel administrativo → Módulo admin/ con dashboards, tablas y gestión de flota.
- RF de reportes → Sección dentro del módulo admin/ para generación de reportes en Word, PDF y Excel.
- RF de notificaciones → Componente global de notificaciones en el layout principal.

Los requerimientos no funcionales también están cubiertos:

- Seguridad: el token JWT se almacena de forma segura y todas las peticiones HTTP incluyen el encabezado de autorización.
- Rendimiento: Vite optimiza el código para producción con lazy loading (carga diferida de módulos), lo que garantiza tiempos de carga rápidos.
- Accesibilidad WCAG 2.1: se usarán atributos ARIA y Tailwind CSS con buenas prácticas de contraste y tamaño de texto.
- PWA: se configurará el manifest.json y el service worker para que la app funcione en Android 13 y 14.

6.2 Relación con mockups e interacciones

Los 30 mockups del Figma se traducen directamente en las páginas y componentes del proyecto:

- Cada pantalla del Figma es una 'page' dentro de su módulo correspondiente.
- Los componentes reutilizables identificados en el análisis (tarjetas, formularios, tablas) se crean en la carpeta components/shared/.
- Las interacciones de navegación del Figma se implementan con React Router.

- Los estados visuales (cargando, error, vacío, lleno) se manejan con los custom hooks y el estado local de React.

6.3 Relación con el modelado

Las entidades del modelado del sistema se reflejan en la estructura del front-end de la siguiente manera:

- Cada entidad tiene su propio servicio en services/ (vehiculosService.js, reservasService.js, pagosService.js, etc.).
- Los datos de cada entidad que necesitan persistir entre páginas se almacenan en el store de Zustand.
- Los formularios del front-end están diseñados para enviar exactamente los datos que el backend espera según el modelado.
- Las relaciones entre entidades (por ejemplo, una reserva está relacionada con un usuario y un vehículo) se manejan combinando datos del estado global y respuestas de la API.

7. Conclusiones

Después de analizar todos los artefactos del proyecto (SRS, mockups del Figma, modelado del sistema e interacciones), se llegó a las siguientes conclusiones:

- React con Vite es la tecnología más adecuada para el proyecto Renta Movil Location porque permite construir tanto la aplicación web como la PWA móvil con una sola base de código, lo cual es eficiente para un equipo de dos personas.
- La arquitectura modular por funcionalidades facilita la división del trabajo y hace el código más fácil de entender y mantener, ya que cada módulo corresponde a un área del sistema bien definida.
- Los patrones de diseño seleccionados (componentes inteligentes/tontos, custom hooks, rutas protegidas y capa de servicios) responden directamente a las necesidades del proyecto: componentes reutilizables para las 30 pantallas identificadas, seguridad con JWT y comunicación ordenada con el backend.
- La estructura de carpetas propuesta garantiza la separación de responsabilidades que exige el SRS (arquitectura en N capas), aplicada al contexto del front-end.

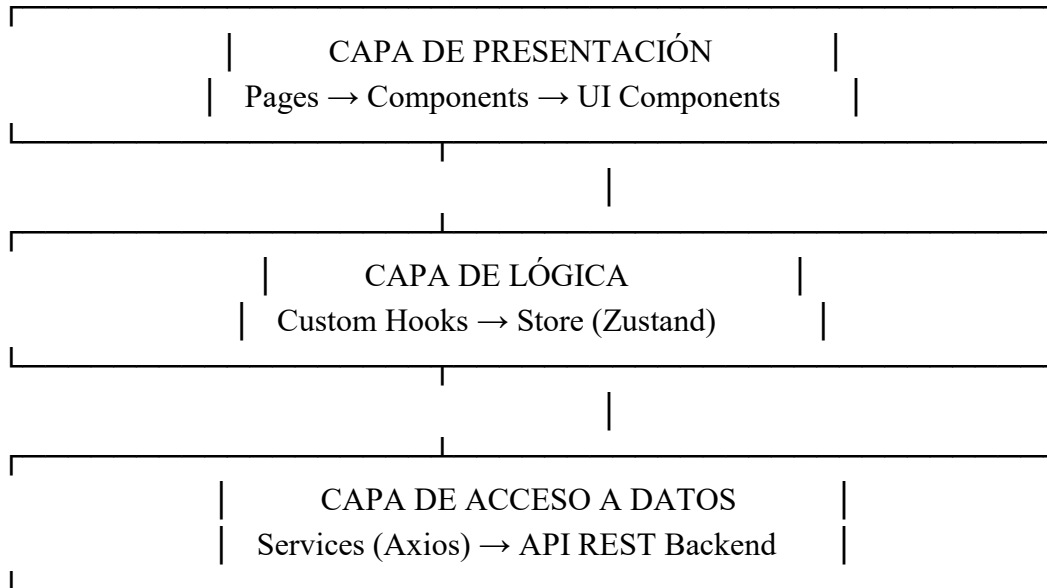
8. Recomendaciones

- Empezar creando los componentes compartidos (botones, inputs, tarjetas) antes de construir los módulos, para que estén disponibles desde el principio.
- Usar Figma como referencia constante durante el desarrollo para garantizar que la implementación corresponda con los mockups aprobados.
- Configurar la PWA desde el inicio del proyecto para evitar problemas de compatibilidad más adelante.
- Documentar cada componente y custom hook con comentarios claros para que cualquier integrante del equipo pueda entenderlo.
- Hacer pruebas frecuentes en dispositivos Android reales para verificar que la experiencia móvil funciona correctamente.
- Mantener la capa de servicios separada de los componentes desde el primer día para facilitar el mantenimiento.

9. Anexos (Opcional)

9.1 Diagrama de arquitectura

El siguiente esquema muestra de forma resumida cómo se relacionan las capas del front-end:



9.2 Stack Tecnológico completo

La estructura completa mostrada en la sección 4.2 sirve como referencia para comenzar el proyecto. Se recomienda crear esta estructura desde el principio, aunque las carpetas empiecen vacías, para que el equipo tenga claro desde el inicio dónde va cada tipo de archivo.

Tecnologías principales (buscar en npm):

- react + react-dom → Base del framework
- vite → Herramienta de construcción (vitejs.dev)
- react-router-dom → Enrutamiento entre páginas
- zustand → Manejo de estado global
- axios → Peticiones HTTP a la API REST
- tailwindcss → Estilos
- vite-plugin-pwa → Configuración PWA para Android

Librería adicional para gestión de fechas y reservas:

- react-big-calendar v1.19 → Calendario visual con vistas de MES, SEMANA y DÍA para mostrar las reservas activas. Permite ver disponibilidad de vehículos de forma visual e intuitiva. Se integra directamente con el hook useReservas() del módulo de reservas.

Comando de instalación: `npm i react-big-calendar`

Referencia: npmjs.com/package/react-big-calendar

- @tanstack/react-query v5 → Manejo de peticiones al servidor, caché de datos y sincronización automática. Evita hacer la misma petición varias veces y mantiene los datos actualizados sin recargar la página. Se combina con Axios en la capa de servicios.

Comando de instalación: `npm i @tanstack/react-query`

Referencia: tanstack.com/query

Resumen de todos los comandos de instalación:

```
npm create vite@latest renta-movil-location -- --template react
npm install react-router-dom
npm install axios
npm install zustand
npm install tailwindcss @tailwindcss/vite
npm i react-big-calendar
npm i @tanstack/react-query
```

9.3 Clasificación de patrones de diseño usados en el proyecto

Los patrones de diseño se dividen en tres categorías. A continuación se muestra cómo los patrones del proyecto Renta Movil Location encajan en esta clasificación:

Patrones Creacionales (cómo se crean los objetos):

No se aplican patrones creacionales clásicos de forma directa en el front-end, ya que React maneja la creación de componentes internamente. Sin embargo, el patrón de composición de componentes (crear componentes complejos combinando componentes simples) cumple una función similar.

Patrones Estructurales (cómo se organizan los elementos):

- Composite: En React, los componentes se anidan dentro de otros componentes formando un árbol. Por ejemplo, la pantalla de reservas contiene un componente Calendario, que contiene celdas de día, que contienen eventos de reserva.

- Facade (Fachada): La capa de servicios (carpeta services/) actúa como fachada. Oculta toda la complejidad de Axios, URLs de la API y manejo de tokens JWT detrás de funciones simples como getVehiculos() o crearReserva().

Patrones de Comportamiento (cómo interactúan los elementos):

- Observer: Zustand implementa este patrón. Cuando el estado global cambia (por ejemplo, se confirma un pago), todos los componentes que están 'observando' ese estado se actualizan automáticamente sin necesidad de hacer nada extra.
- Strategy: Las rutas protegidas implementan este patrón. Según el rol del usuario (cliente, administrador, operador), la estrategia de navegación cambia: cada rol ve rutas y pantallas diferentes.
- Template Method: Los custom hooks (useReservas, useVehiculos, useAuth) siguen una estructura base similar: obtener datos → manejar carga → manejar error → retornar resultado. Cada hook personaliza los pasos según su necesidad específica.